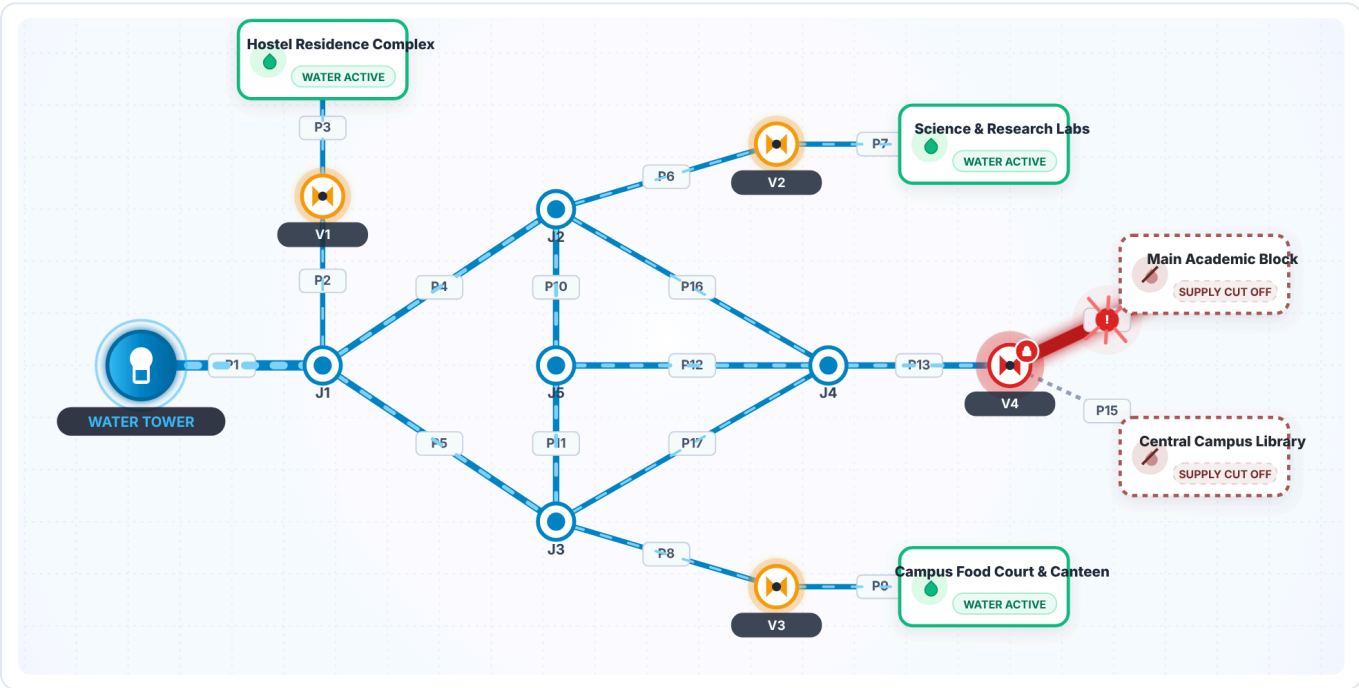
SIMPLE GUIDE · FUNDAMENTALS OF AI MICRO-PROJECT

Campus Water Network: Pipe-Burst Isolation with BFS

What the website does, how it works step by step, and how the code is organised.



The website after a burst on pipe P14: valve V4 is closed and two buildings are cut off.

Institution: Karunya Institute of Technology and Sciences
Course: Fundamentals of Artificial Intelligence (25DC204)
Academic year: 2026–2027

Team: Dabria Susan Sam · G Revanth Hari · Sujith S. · Leah Sasha V.L.
Live site: kits-campus-water-bfs.netlify.app
Code: github.com/KrisJeffordImmanuvel/FAI_Sujith

1 What does this website do?

Imagine the water system of our campus: one **water tower**, a set of **pipes**, some **valves** that can be shut, and five **buildings** that need water.

Now imagine one pipe bursts. Two questions need answers quickly:

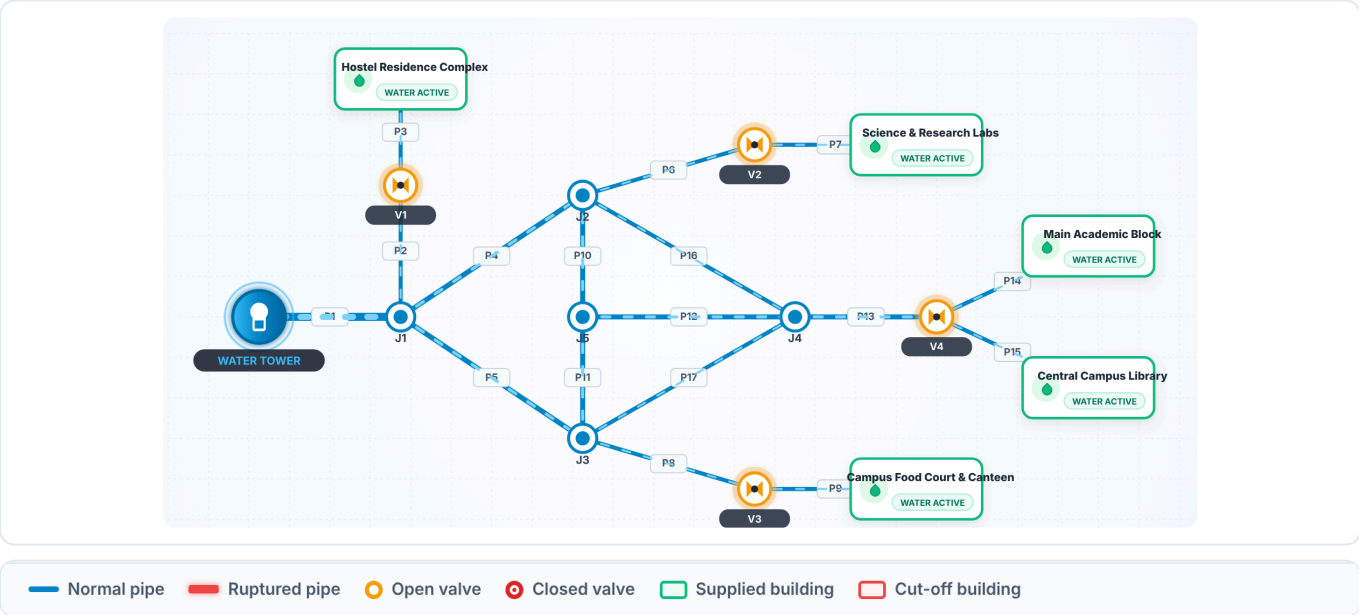
1. **Which valves should we close** to stop the leak?
2. **Which buildings will lose water** once those valves are closed?

The website answers both questions automatically using an algorithm called **Breadth-First Search (BFS)**, and it animates every step so you can see exactly how the answer was found.

> **In one sentence**
>
> You click a pipe to "break" it; BFS finds the nearest valves to close, then checks which buildings can still be reached from the water tower, and the site shows the result on a map and in a printable report.

2 The campus network

To a computer, the water system is a **graph**: a set of dots (called *nodes*) joined by lines (called *edges*). Every pipe is an edge; everything else is a node.



The network when nothing is broken, and the legend shown under the map on the website.

Part	How many	What it is
Water tower (WT)	1	Where all the water comes from.
Junctions (J1–J5)	5	Points where pipes meet and split.
Valves (V1–V4)	4	Gates that can be shut. There is one in front of each building group.
Buildings	5	Hostel, Research Labs, Food Court & Canteen, Academic Block, Library.
Pipes (P1–P17)	17	The connections. Some form *loops*, so there can be more than one route between two points.

3 What is BFS? (the simple version)

Think of dropping a stone into a pond. The ripple spreads out in rings: first the nearest water, then the next ring, then the next. **BFS explores a graph the same way**: first everything one pipe away, then everything two pipes away, and so on.

Ring 1
Nodes 1 pipe away from the start

Ring 2
Nodes 2 pipes away

Ring 3 ...
Keeps going until there is nothing new to visit

To keep the rings in order, BFS uses a **queue**. It works like the line at the canteen counter: whoever joins first is served first ("first in, first out"). BFS also keeps a **visited list** so it never checks the same node twice, even when pipes form a loop.

> **The key idea**
>
> BFS counts **hops** (how many pipes you cross), not metres. BFS spreads out from a burst pipe and stops at the first valve it meets on each branch. Of those valves, only the ones on the **water-tower side** matter: water reaches the burst through them. Those are **the nearest valves that isolate the rupture (fewest hops)**. Closing any other valve would be unnecessary.

The website uses BFS **twice**:

- **1st BFS:** starts at the burst pipe and spreads out until it hits valves, then keeps only the valves on the water-tower side. Those are the valves to close. If the search reaches the water tower without passing any valve, no valve can isolate that pipe, so none is closed.
- **2nd BFS:** starts at the water tower and spreads through the open pipes. Any building it reaches still has water; any building it cannot reach is cut off.

4 How it works: the 5 steps

1. **Select pipe** — mark the burst
2. **Find boundary valves** — 1st BFS
3. **Close valves** — shut them
4. **Check supply** — 2nd BFS
5. **Report** — save as PDF

1 Select pipe

Click a pipe on the map, or choose it from the list. This is the pipe that “burst”.

You see: the pipe turns red with a burst symbol.

2 Find boundary valves (1st BFS)

BFS starts from *both ends* of the burst pipe at the same time and spreads outwards one pipe at a time. It never crosses the burst pipe itself. Whenever it reaches a valve, it notes that valve and does not go past it. At the end it keeps only the valves on the **water-tower side** (the ones water would flow through to reach the burst). If the search reaches the **water tower** without passing a valve, it stops: no valve stands between the tower and the burst, so valves cannot isolate this pipe.

You see: the queue, the visited nodes and the order of visits update live; the valves to close glow orange.

3 Close valves

The valves found in Step 2 are shut, and the burst pipe is now cut off from the water tower. If Step 2 found that no valve can isolate the pipe, nothing is closed, because closing other valves would not stop the leak and would only cut off buildings.

You see: closed valves turn red with a small lock (or a message saying nothing could be closed).

4 Check supply (2nd BFS)

BFS now starts at the **water tower**. It moves through open pipes only: it cannot pass a closed valve or the burst pipe. Every building it reaches is **supplied**; every building it cannot reach is **cut off**.

You see: supplied buildings in green (“WATER ACTIVE”), cut-off buildings in red (“SUPPLY CUT OFF”).

5 Report

A one-page summary: which pipe broke, which valves were closed, which buildings are affected, and the BFS visit order. Click **Download / Print PDF** to save it.

Shortcut

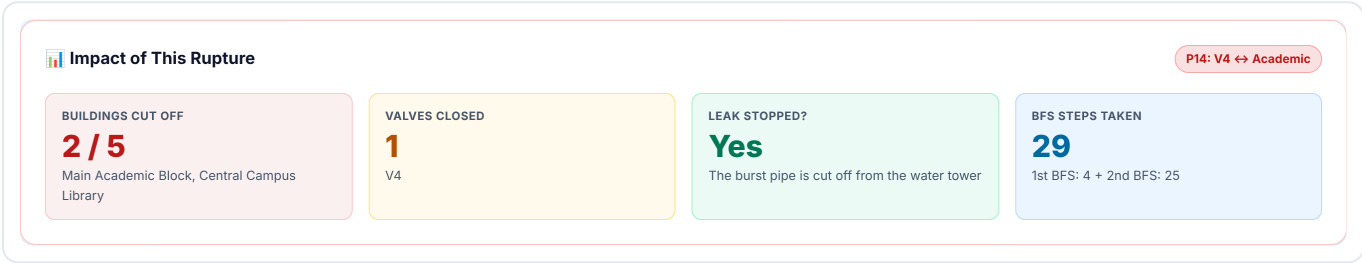
⚡ **Full Simulation** runs all 5 steps automatically. While a step is animating you can use **Play/Pause**, **Next**, **Prev** and **Reset** (keyboard: Space, →, ←, R).

5 A worked example: pipe P14 bursts

P14 is the short pipe from valve **V4** to the **Academic Block**.

P14 bursts → End 1: V4 is a valve → close it + End 2: Academic Block has no other pipes → stop

1. **1st BFS:** the two ends are V4 and the Academic Block. V4 is already a valve, so it goes straight onto the list. The Academic Block has no other pipes, so there is nowhere else to go. V4 is on the water-tower side (water reaches the burst through it). Result: **close V4 only**.
2. **Close valves:** V4 is shut.
3. **2nd BFS:** from the water tower, water reaches J1 → J2, J3, V1 ... and every building *except* the two that sit behind V4.
4. **Result:** 2 of 5 buildings are cut off: the Academic Block *and* the Library. The Library loses water too, even though its own pipe is fine, because it shares valve V4.



The “Impact of This Rupture” panel that appears after every simulation.

The four ready-made scenarios on the Dashboard

Scenario	Pipe	Valves closed	Buildings cut off	Leak stopped?	Why
Water tower trunk (worst case)	P1	None	All 5	No	P1 is the only pipe leaving the tower and has no valve.
Hostel service line (single building)	P3	V1	Hostel	Yes	The Hostel is a dead end behind V1.
Academic lateral (shared valve)	P14	V4	Academic, Library	Yes	Both buildings sit behind V4.
Bypass loop (looped pipe)	P16	None	None	No	See the box below.

Why can't a burst on a loop pipe be isolated?

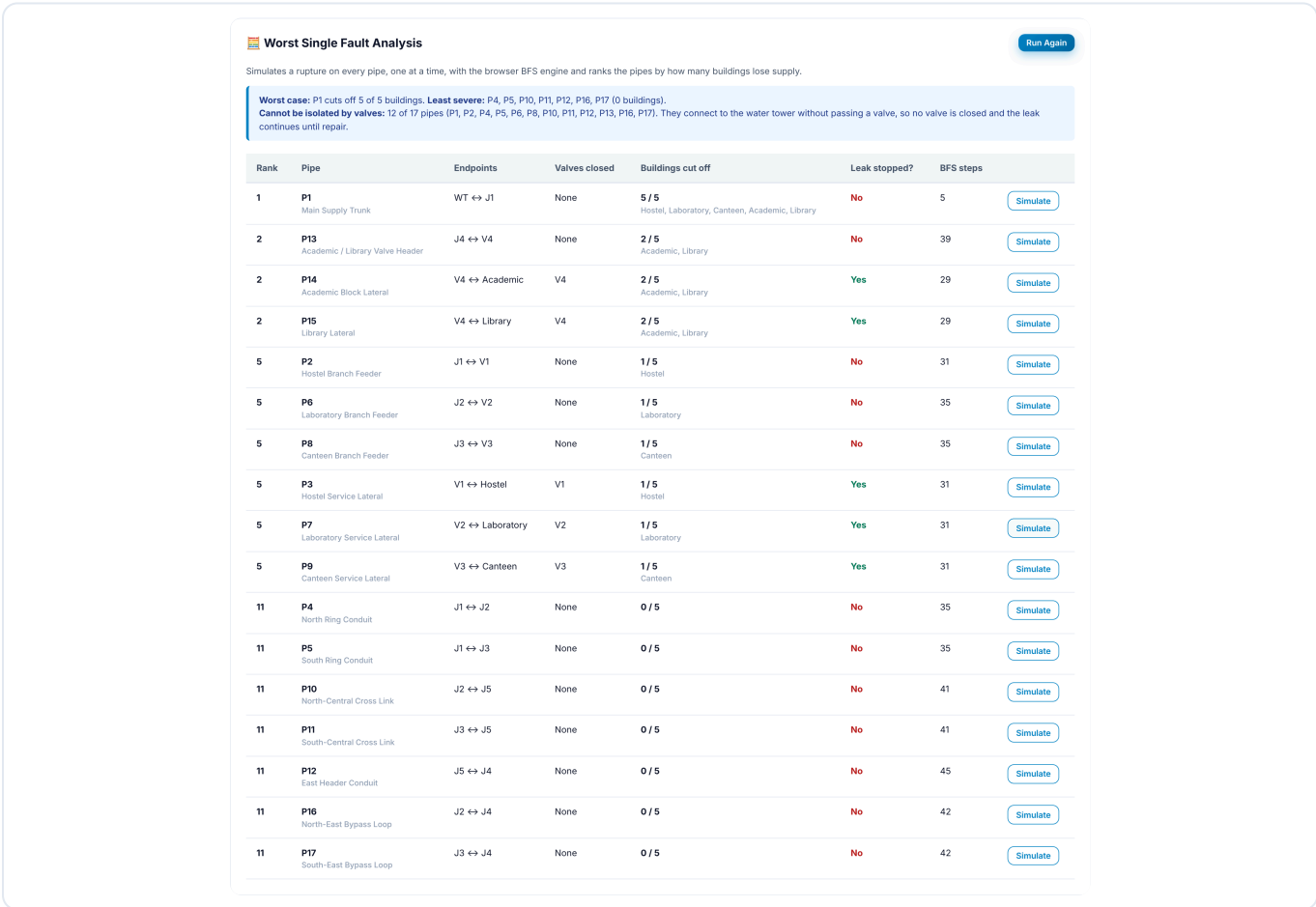
When P16 breaks, water reroutes around the loop (through J5), so every building keeps its supply. But the loop pipes have no valves of their own: starting from J2, BFS reaches the water tower without passing a valve. So no valve stands between the tower and the burst. Closing the building valves would not stop the leak; it would only cut off buildings. That's why nothing is closed, and the leak continues until the pipe is repaired. Adding valves on the loop pipes would make these bursts isolatable. This is a real lesson about where valves should be installed.

6 A quick tour of the website

Tab	What you can do there
Dashboard	Network summary, the four one-click scenarios, and the *Worst Single Fault* analysis.
Interactive Simulation	The map. Break a pipe, follow Steps 1–5, watch the queue and visited list, see the impact panel.
Algorithm Visualizer	A closer look at the BFS queue, the node being expanded, and a step-by-step event log.
BFS Algorithm Theory	The graph model, what BFS finds (and what it doesn't), and pseudocode for both BFS passes.
Isolation Report	The printable report for the last simulation, with the *Download / Print PDF* button.
About & Team	Course details, team roles and a short disclaimer.

Worst Single Fault

One button breaks each of the 17 pipes in turn and ranks them by how many buildings lose water. It also shows whether the leak can be stopped with valves. Each row has a *Simulate* button to replay that case on the map.



P1 is the worst case. 12 of the 17 pipes cannot be isolated by valves, because the only valves are in front of buildings.

7 How the website is built

It is a plain website: **HTML** for the page, **CSS** for the look, and **JavaScript** for everything that moves. There is no framework and nothing to "build". An optional small **Python** server can run the same BFS on the server side.

File	What it does
`index.html`	The page: tabs, cards, buttons and text.
`css/style.css`, `css/animations.css`	Colours, layout, animations, phone layout and print layout.
`js/network-data.js`	The campus network: every node, every pipe, and the four scenarios.
`js/bfs.js`	**The BFS algorithm** (1st and 2nd pass) in JavaScript. It also records every step for the animation.
`js/simulation.js`	Runs Steps 1–5, Play/Pause/Next/Prev, and talks to the Python server when chosen.
`js/visualizer.js`	Draws the map (pipes, valves, buildings, colours).
`js/report.js`	Builds the Isolation Report.
`js/app.js`	Connects everything: tabs, buttons, panels, scenarios, worst-fault table.
`backend/app.py`	Small Python web server: serves the website and answers API requests.
`backend/bfs_engine.py`	The same BFS written in Python.
`requirements.txt`	List of extra Python packages. It is empty on purpose: only built-in Python is used.

Two engines, same answer

DEFAULT

Browser engine

The BFS in `js/bfs.js` runs inside your browser. It needs no server and works even on a basic static host.

OPTIONAL

Python engine

Choose *Python REST API* under "Algorithm Engine". The browser sends the burst pipe to `backend/app.py`, which runs `bfs_engine.py` and sends the steps back. If the server can't be reached, the site says *"Server not reachable – using browser engine"* and carries on.

Both engines were checked on all 17 pipes and give identical results.

Run it on your computer

1. Install Python 3 (nothing else is needed).
2. Open a terminal in the project folder and run: `python backend/app.py`
3. Open `http://localhost:5000` in your browser. To open it on a phone, connect the phone to the same Wi-Fi and use the “Phone/Mobile” address the server prints.

8 Good to know (limitations)

- **It is a connection model, not a hydraulic simulation.** It does not calculate water pressure or flow. “Supplied” simply means a building is still connected to the water tower through open pipes.
- **Nearest means fewest pipes, not shortest distance.** BFS ignores pipe length.
- **Valve placement decides the outcome.** With valves only in front of buildings, a burst on the tower pipe, a junction pipe or a loop pipe cannot be isolated by valves; the leak continues until the pipe is repaired.
- **The water-saved figure in the report is a rough illustration**, not a measured value.